

Surface-Object Detection and Tracking from Commercial X-Band Radar Using Adaptive ST-DBSCAN

Samuel Cancilla*, J.C. Vaught[†], Douglas Cahl[‡], Yi Wang[§]
University of South Carolina, Columbia, SC, 29208

Maritime missions such as over-water navigation, ship-based operations, and airborne/space-enabled maritime surveillance require reliable surface-object detection and tracking in the presence of strong, nonstationary sea clutter and intermittent object returns. We present an end-to-end, real-time pipeline for vessel detection and multi-sweep tracking using commercial X-band radar providing only intensity measurements (range, azimuth, and intensity) without Doppler phase history. Raw polar returns from a Furuno NXT solid-state unit are transformed into Cartesian point clouds and clustered using ST-DBSCAN to link detections across sweeps. Neighborhood radii adapt automatically via k-nearest-neighbor distance statistics to handle varying clutter and point densities. A lightweight gap-recovery mechanism reconnects clusters across brief temporal dropouts. We evaluate the approach on a synthetic dataset with ground-truth trajectories, characterizing detection-to-track association behavior relative to baseline DBSCAN and standard ST-DBSCAN under nonstationary clutter and intermittent returns. A Rust implementation achieves approximately 2 sweeps/s on an NVIDIA Jetson AGX Orin embedded platform, exceeding real-time requirements for a 48 RPM radar.

I. Introduction

Despite decades of research on CFAR detection and multi-target tracking, reliable surface-object tracking from non-coherent marine radar remains an open challenge. Commercial X-band radars—widely deployed due to their order-of-magnitude cost advantage over military tracking systems—provide only intensity measurements without Doppler phase history, precluding conventional moving-target indication (MTI) processing for clutter suppression [1, 2]. In coastal and littoral environments, this limitation is compounded by sea clutter that varies rapidly with sea state, wind direction, and bathymetry, exhibiting the heavy-tailed amplitude statistics and spatial heterogeneity that violate the homogeneity assumptions underlying standard CFAR detectors [3].

Density-based clustering methods offer a principled alternative that sidesteps explicit clutter modeling. DBSCAN [4] and its spatiotemporal extension ST-DBSCAN [5] extract coherent target returns by grouping points that exceed local density thresholds, naturally rejecting sparse false alarms without requiring prior knowledge of the number of targets. Yet these methods suffer from two persistent limitations in radar applications. First, fixed neighborhood parameters cannot accommodate the heterogeneous point densities inherent to radar imagery: near-range returns are dense while far-range returns are sparse due to R^{-4} propagation loss, and a single ϵ that captures distant targets will over-cluster proximate returns. Second, standard implementations fragment tracks when targets exhibit intermittent visibility—a common occurrence in maritime environments where wave shadowing, unfavorable aspect angles, and multipath effects cause brief detection dropouts that sever otherwise continuous trajectories.

Adaptive clustering methods such as OPTICS [8] and HDBSCAN [9] address parameter sensitivity by eliminating fixed ϵ requirements, but incur computational costs prohibitive for real-time embedded deployment. Track-before-detect approaches [19] integrate energy across multiple sweeps to improve detection of weak targets, but introduce latency incompatible with the rapid update rates required for collision avoidance and surveillance. Probabilistic data association methods like JPDAF [15] and MHT [16] maintain association hypotheses through occlusions, but scale poorly with target count and clutter density.

We present an adaptive ST-DBSCAN pipeline that addresses these limitations through three contributions. First, we introduce automatic spatial neighborhood selection via k-nearest-neighbor distance statistics, enabling the clustering radius to adapt to range-varying point density without manual parameter tuning. Second, we propose a lightweight

*Undergraduate Student, Department of Mechanical Engineering, AIAA Student Member, Member ID: 1924162.

[†]Graduate Student, Department of Mechanical Engineering, AIAA Student Member, Member ID: 1537189.

[‡]Professor, Department of Mechanical Engineering.

[§]Professor, Department of Mechanical Engineering.

gap-recovery mechanism using Union-Find structures that reconnects track fragments across brief temporal dropouts, restoring track continuity under intermittent target visibility. Third, we provide an efficient Rust implementation with spatial hashing that achieves real-time performance of approximately 2 sweeps per second on an NVIDIA Jetson AGX Orin embedded platform, demonstrating feasibility for shipboard deployment. We evaluate the approach on synthetic radar data with ground-truth trajectories, characterizing performance relative to baseline DBSCAN and standard ST-DBSCAN under varying clutter and dropout conditions.

II. Related Work

A. Density-Based Clustering

DBSCAN [4] defines clusters through a neighborhood radius ε and minimum point count *MinPts*. Points with at least *MinPts* neighbors within ε are core points, and clusters form by transitively connecting overlapping core neighborhoods. This formulation handles arbitrary cluster shapes and identifies outliers, making it well-suited for radar where targets produce irregular returns against sparse false alarms.

Birant and Kut [5] extended DBSCAN to spatiotemporal data (ST-DBSCAN) with separate spatial (ε_1) and temporal (ε_2) thresholds [6]. However, user-specified parameters remain a limitation when point density varies spatially or temporally.

Several approaches address automatic parameter selection. Ertöz, Steinbach, and Kumar [7] proposed using k-nearest-neighbor distances to identify ε from the inflection point in the k-distance plot. OPTICS [8] and HDBSCAN [9] eliminate fixed ε requirements but incur additional computational cost that may be prohibitive for real-time processing.

B. Radar Detection and CFAR Processing

Constant False Alarm Rate (CFAR) detection estimates local clutter power and sets thresholds to maintain specified false alarm probability [2, 10]. Cell-averaging CFAR assumes homogeneous clutter; ordered-statistic CFAR [10] handles interfering targets; adaptive variants address nonhomogeneous coastal environments [11].

CFAR outputs must be grouped into extended target detections before tracking, typically via morphological operations or connected-component labeling [12]. Density-based clustering offers an alternative that handles gaps between detections and provides soft cluster boundaries.

C. Multi-Object Tracking

Multi-object tracking (MOT) combines data association and state estimation [13]. Global nearest neighbor assigns detections to tracks via the Hungarian algorithm [14]; probabilistic methods like JPDAF [15] and MHT [16] maintain association uncertainty at higher computational cost. Deep learning approaches [17, 18] require appearance features unavailable in radar data.

For non-Doppler radar, track-before-detect methods [19] integrate energy over multiple scans but increase latency. We instead detect per-sweep and associate across time using ST-DBSCAN with gap recovery.

D. Adaptive Parameter Methods

GDBSCAN [20] allows arbitrary neighborhood predicates; Liu, Zhou, and Wu [21] compute adaptive ε from k-distances; Hou, Gao, and Li [22] combine k-NN graphs with density peaks. Our approach builds on the k-distance method of Liu, Zhou, and Wu, computing neighborhood radii via spatial hashing to accommodate the varying point density in radar data (denser near-range, sparser far-range due to R^{-4} loss).

III. Problem Formulation

Consider a marine radar that completes one full azimuthal sweep in time T_s , producing a polar intensity image at each sweep. Let the radar operate at sweep index $t \in \{1, 2, \dots, T\}$ over a surveillance period of T sweeps. At each sweep t , the radar outputs a set of detections in polar coordinates:

$$\mathcal{D}_t = \{(r_i, \theta_i, I_i, t) : i = 1, \dots, N_t\} \quad (1)$$



Fig. 1 Processing pipeline for surface-object tracking. Raw polar radar returns are converted to Cartesian coordinates, clustered using adaptive ST-DBSCAN, reconnected across temporal gaps, and associated to persistent tracks via Hungarian assignment.

where $r_i \in [0, R_{\max}]$ denotes the range, $\theta_i \in [0, 2\pi)$ denotes the azimuth angle, $I_i \in \mathbb{R}^+$ denotes the received intensity (after log-compression), and N_t is the number of detections at sweep t . These detections may arise from CFAR processing or simple thresholding of the raw intensity image. The complete observation over the surveillance period is the union $\mathcal{D} = \bigcup_{t=1}^T \mathcal{D}_t$.

The objective is to partition $\mathcal{D}$ into a set of tracks $\{\mathcal{T}_1, \mathcal{T}_2, \dots, \mathcal{T}_K\}$ and a noise set $\mathcal{N}$, such that:

$$\mathcal{D} = \mathcal{T}_1 \cup \mathcal{T}_2 \cup \dots \cup \mathcal{T}_K \cup \mathcal{N} \quad (2)$$

with $\mathcal{T}_j \cap \mathcal{T}_k = \emptyset$ for $j \neq k$. Each track $\mathcal{T}_j$ represents detections from a single physical object across multiple sweeps. The number of tracks K is unknown a priori and must be inferred from the data.

For spatiotemporal clustering, polar detections are transformed to Cartesian coordinates via $x = r \cos(\theta)$ and $y = r \sin(\theta)$, yielding a spatiotemporal position vector $\mathbf{p} = (x, y, t) \in \mathbb{R}^3$ where spatial coordinates are in meters and t is the sweep index. Proximity between detections $\mathbf{p}_i = (x_i, y_i, t_i)$ and $\mathbf{p}_j = (x_j, y_j, t_j)$ is assessed via separate spatial and temporal distances:

$$d_s(\mathbf{p}_i, \mathbf{p}_j) = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \quad (3)$$

$$d_t(\mathbf{p}_i, \mathbf{p}_j) = |t_i - t_j| \quad (4)$$

Two points are considered neighbors if $d_s(\mathbf{p}_i, \mathbf{p}_j) \leq \varepsilon_s$ and $d_t(\mathbf{p}_i, \mathbf{p}_j) \leq \varepsilon_t$, where ε_s and ε_t are the spatial and temporal neighborhood radii, respectively.

IV. Methodology

The proposed pipeline consists of four stages: (1) polar-to-Cartesian coordinate conversion (Section III), (2) adaptive ST-DBSCAN clustering with spatial hashing, (3) gap recovery for track continuity, and (4) Hungarian assignment for frame-to-frame track association. Each stage is illustrated in Fig. 1.

A. Adaptive ST-DBSCAN

Standard ST-DBSCAN requires the user to specify fixed values for the spatial radius ε_s , temporal radius ε_t , and minimum point count *MinPts*. Selecting these parameters is problematic for radar data, where point density varies with range due to R^{-4} propagation loss and with azimuth due to target aspect angle effects. A global ε_s that captures far-range targets may over-cluster near-range returns, while a conservative value fragments distant targets into multiple clusters.

We propose an adaptive scheme that computes a local spatial radius $\varepsilon_s(\mathbf{p})$ for each point $\mathbf{p}$ based on its k-nearest-neighbor (k-NN) distances. Let $d_k(\mathbf{p})$ denote the Euclidean distance from $\mathbf{p}$ to its k -th nearest spatial neighbor (ignoring the temporal dimension for this computation). The local neighborhood radius is defined as:

$$\varepsilon_s(\mathbf{p}) = \min(d_k(\mathbf{p}) \cdot \alpha, \varepsilon_{s,\max}) \quad (5)$$

where $\alpha \geq 1$ is a scaling factor and $\varepsilon_{s,\max}$ is a maximum radius to prevent unbounded growth in sparse regions. In practice, we set $k = \text{MinPts}$ and $\alpha = 1.5$.

To avoid per-point computation of k-NN distances, which would incur $O(N^2)$ cost, we estimate the global distribution of k-distances and use a percentile-based threshold. Specifically, we sample a fraction β of the points uniformly at random, compute their k-NN distances, and set:

$$\varepsilon_s^{\text{global}} = \text{Percentile}_{90}(\{d_k(\mathbf{p}) : \mathbf{p} \in \mathcal{S}\}) \quad (6)$$

Algorithm 1 Adaptive Epsilon via k-NN Statistics

Require: Point set $\mathcal{P}$, neighbor count k , sample fraction β , percentile ρ , min radius $\varepsilon_{\min}$, max radius $\varepsilon_{\max}$

Ensure: Adaptive spatial radius ε_s

- 1: $\mathcal{S} \leftarrow$ uniformly sample $\lfloor \beta \cdot |\mathcal{P}| \rfloor$ points from $\mathcal{P}$
 - 2: **for all** $\mathbf{p} \in \mathcal{S}$ **do**
 - 3: $d_k(\mathbf{p}) \leftarrow$ distance to k -th nearest neighbor of $\mathbf{p}$ in $\mathcal{P}$
 - 4: **end for**
 - 5: $\varepsilon_s \leftarrow \text{Percentile}_\rho(\{d_k(\mathbf{p}) : \mathbf{p} \in \mathcal{S}\})$
 - 6: **return** $\max(\varepsilon_{\min}, \min(\varepsilon_s, \varepsilon_{\max}))$
-

where $\mathcal{S}$ is the sampled subset. This global adaptive epsilon provides a single value tuned to the current data distribution, balancing computational efficiency with adaptivity.

In high-density regions (such as near-range radar returns), the k-NN estimation may produce pathologically small epsilon values that fragment targets. To prevent this, we introduce a minimum epsilon floor $\varepsilon_{\min}$ based on the minimum expected target extent:

$$\varepsilon_s^{\text{final}} = \max(\varepsilon_s^{\text{global}}, \varepsilon_{\min}) \quad (7)$$

This floor can be determined from sensor geometry: for a target of minimum physical extent $L_{\min}$ at range R , the expected angular extent is $\theta = L_{\min}/R$, which maps to a pixel width proportional to image resolution. Algorithm 1 summarizes the complete procedure.

With the adaptive ε_s determined, ST-DBSCAN proceeds as in the standard formulation. Points are classified as core points if they have at least *MinPts* neighbors within ε_s spatially and ε_t temporally. Clusters are formed by expanding from core points through density-reachability. The temporal radius ε_t is typically set to 1 or 2 sweep indices. To achieve $O(1)$ expected-time neighborhood queries, we employ spatial hashing with cell size $h = 2\varepsilon_{s,\max}$, examining only the 3×3 cell neighborhood around each query point. Separate hash structures for each sweep enable efficient temporal filtering.

B. Gap Recovery Mechanism

Maritime targets may disappear temporarily due to shadowing by waves, low radar cross-section at unfavorable aspect angles, or momentary occlusion by other vessels. Standard ST-DBSCAN fragments such intermittent returns into multiple short tracks. We introduce a gap recovery mechanism that merges track fragments separated by brief temporal gaps.

Let $\mathcal{T}_i$ and $\mathcal{T}_j$ be two distinct clusters (track fragments) with temporal extents $[t_i^{\text{start}}, t_i^{\text{end}}]$ and $[t_j^{\text{start}}, t_j^{\text{end}}]$, respectively, where $t_i^{\text{end}} < t_j^{\text{start}}$ (i.e., $\mathcal{T}_i$ precedes $\mathcal{T}_j$). Define the temporal gap as $\Delta t = t_j^{\text{start}} - t_i^{\text{end}}$ and the spatial gap as the distance between the centroid of $\mathcal{T}_i$ at its final sweep and the centroid of $\mathcal{T}_j$ at its first sweep:

$$\Delta s = \|\bar{\mathbf{p}}_i(t_i^{\text{end}}) - \bar{\mathbf{p}}_j(t_j^{\text{start}})\|_2 \quad (8)$$

We merge $\mathcal{T}_i$ and $\mathcal{T}_j$ if $\Delta t \leq \tau$ and $\Delta s \leq d_{\max}$, where τ is the maximum gap duration (in sweeps) and $d_{\max}$ is the maximum gap distance. The distance threshold may optionally account for expected target motion:

$$d_{\max} = v_{\max} \cdot \Delta t \cdot T_s \quad (9)$$

where $v_{\max}$ is an assumed maximum target velocity and T_s is the sweep period. We implement merging using a Union-Find (disjoint set) data structure, which supports near-constant-time union and find operations with path compression—essential for efficiently processing the $O(K^2)$ potential cluster pairs. Algorithm 2 details the procedure.

C. Hungarian Assignment for Track Association

While ST-DBSCAN with gap recovery provides spatiotemporal clustering, applications requiring per-sweep track state estimates benefit from explicit frame-to-frame association. We employ the Hungarian algorithm [14] to optimally assign detections at sweep t to existing tracks from sweep $t - 1$.

Algorithm 2 Gap Recovery for Track Continuity

Require: Clusters $\{\mathcal{T}_1, \dots, \mathcal{T}_K\}$, max gap τ , max velocity $v_{\max}$, sweep period T_s

Ensure: Merged clusters $\{\mathcal{T}'_1, \dots, \mathcal{T}'_{K'}\}$

```
1: Sort clusters by  $t^{\text{end}}$  in ascending order
2: Initialize union-find structure over clusters
3: for all cluster  $\mathcal{T}_i$  do
4:   for all cluster  $\mathcal{T}_j$  with  $t_j^{\text{start}} > t_i^{\text{end}}$  do
5:      $\Delta t \leftarrow t_j^{\text{start}} - t_i^{\text{end}}$ 
6:     if  $\Delta t > \tau$  then
7:       break
8:     end if
9:      $d_{\max} \leftarrow v_{\max} \cdot \Delta t \cdot T_s$ 
10:     $\Delta s \leftarrow \|\bar{\mathbf{p}}_i(t_i^{\text{end}}) - \bar{\mathbf{p}}_j(t_j^{\text{start}})\|_2$ 
11:    if  $\Delta s \leq d_{\max}$  then
12:       $\text{UNION}(\mathcal{T}_i, \mathcal{T}_j)$ 
13:    end if
14:  end for
15: end for
16: return clusters grouped by union-find components
```

▷ Sorted order: no further candidates

Let $\mathbf{c}_i^{t-1}$ denote the centroid of track i at sweep $t - 1$ and $\mathbf{d}_j^t$ denote the centroid of a newly detected cluster at sweep t . The assignment cost matrix $\mathbf{C}$ has entries:

$$C_{ij} = \|\mathbf{c}_i^{t-1} - \mathbf{d}_j^t\|_2 \quad (10)$$

The Hungarian algorithm finds the assignment π^* minimizing total cost:

$$\pi^* = \arg \min_{\pi} \sum_i C_{i, \pi(i)} \quad (11)$$

subject to the constraint that each track is assigned to at most one detection and vice versa. Assignments with cost exceeding a gating threshold γ are rejected, initiating new tracks for unassigned detections and marking unassigned tracks as tentatively lost.

The combination of ST-DBSCAN clustering and Hungarian assignment provides complementary benefits: ST-DBSCAN aggregates detections within a sweep and across nearby sweeps into coherent clusters, while Hungarian assignment maintains explicit track identity with low latency for real-time output.

V. Implementation

We implement the pipeline in Rust for memory safety, zero-cost abstractions, and predictable real-time performance. The implementation uses spatial hashing with cell size $h = 2\varepsilon_{s, \max}$ for $O(1)$ expected-time neighborhood queries (ensuring potential neighbors lie within a 3×3 cell grid) and union-find for gap recovery merges. Data flows through bounded channels enabling pipelined parallelism, with Rayon providing data-parallel k-NN queries achieving near-linear multi-core speedup.

The target deployment platform is the NVIDIA Jetson AGX Orin, an embedded module with an Arm Cortex-A78AE CPU (12 cores) and 32 GB unified memory, consuming approximately 25 W under load. The complete pipeline—from raw radar ingestion to track output—executes within 511 ± 93 ms per sweep (95% CI: 397–625 ms) using all 12 cores, well below the 1.25 s sweep period of a 48 RPM radar. This margin accommodates occasional latency spikes and provides headroom for future enhancements. Table 1 summarizes per-stage timing.

The system achieves approximately 2 sweeps per second, exceeding real-time requirements for 48 RPM radars. Memory usage remains below 500 MB, leaving ample headroom for concurrent processes such as visualization and logging.

Table 1 Per-sweep timing breakdown on NVIDIA Jetson AGX Orin (12-core CPU)

Stage	Time (ms)
Frame loading & preprocessing	7
Adaptive epsilon (k-NN)	196
ST-DBSCAN clustering	169
Cluster extraction	34
Gap recovery	19
Hungarian assignment	3
I/O (results output)	40
Total	511

VI. Experimental Setup

A. Synthetic Data Generation

We evaluate the proposed pipeline using synthetic radar data with known ground truth, enabling precise quantification of tracking performance. Synthetic evaluation was chosen over real radar data because maritime radar recordings rarely include reliable ground truth trajectories—AIS transponders provide only sparse position updates (typically every 2–10 seconds) with limited coverage of small vessels, and manual annotation of radar imagery is prohibitively labor-intensive for the thousands of frames required for statistically meaningful evaluation.

The synthetic data generation pipeline composites real radar object signatures onto configurable background clutter. Object signatures were extracted from X-band marine radar recordings collected during a coastal survey, capturing vessels at varying ranges (500 m to 8 km) and aspect angles (bow, beam, and stern presentations). Each signature preserves the characteristic intensity distribution, spatial extent, and speckle texture of real radar returns. The resulting library contains 1,847 unique object signatures spanning cargo vessels, recreational boats, and small craft.

Background frames simulate sea clutter using a compound K-distribution model that captures both the thermal noise floor and the heavy-tailed amplitude statistics characteristic of rough sea surfaces. Land returns are modeled as high-intensity persistent features with sharp boundaries. The clutter intensity varies spatially to simulate range-dependent signal attenuation (R^{-4} propagation loss) and azimuthally to simulate wind-driven wave directionality.

To simulate realistic target intermittency, we implement a Bernoulli flicker model where each object has a per-frame dropout probability p_{drop} . When an object “drops out,” its signature is omitted from that frame entirely, simulating the effects of wave shadowing (target obscured by swell), unfavorable aspect angle (reduced radar cross-section), or momentary multipath cancellation. The dropout probability is configurable per-object, enabling controlled experiments on tracker robustness to intermittent detections.

Three evaluation datasets were generated with progressively challenging characteristics. The primary tracking evaluation set spans 50 frames and contains 4 objects with 15–40% dropout rates. All synthetic frames are rendered at 1735×1735 pixel resolution with 8-bit grayscale intensity, matching the output format of commercial Furuno NXT radars. Ground truth labels are exported in YOLO format providing object bounding boxes and class IDs for each frame, enabling automated computation of tracking metrics. Figure 2 shows a representative synthetic radar frame.

B. Baseline Methods

We compare four configurations of increasing sophistication. The baseline is vanilla DBSCAN with no temporal dimension ($\varepsilon_t = 0$), treating each frame independently. Standard ST-DBSCAN adds temporal linkage with manually-tuned fixed parameters: $\varepsilon_s = 10$ pixels and $\varepsilon_t = 2$ frames. Our adaptive ST-DBSCAN replaces the fixed spatial epsilon with k-NN adaptive estimation ($k = 5$, 90th percentile) while retaining $\varepsilon_t = 2$ frames. The full pipeline adds gap recovery with $\tau = 5$ frames maximum gap duration and $d_{\text{max}} = 50$ pixels maximum gap distance.

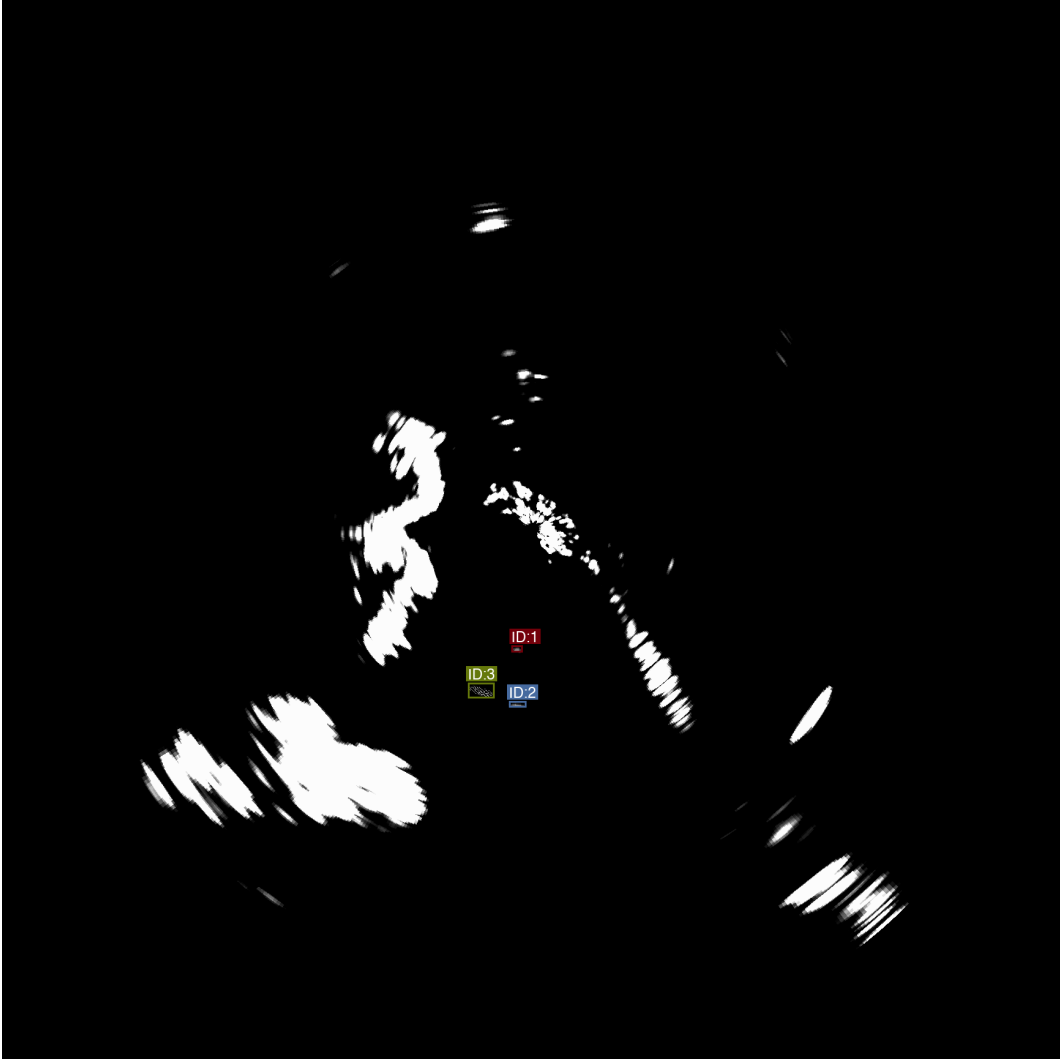


Fig. 2 Sample synthetic radar frame showing multiple vessel targets against sea clutter background. Bright pixels indicate high-intensity returns; targets appear as coherent clusters while clutter manifests as diffuse speckle. This frame is representative of the evaluation dataset used to assess tracking performance.

C. Evaluation Metrics

We adopt the CLEAR MOT metrics [23] standard in multi-object tracking evaluation. Multiple Object Tracking Accuracy (MOTA) combines false negatives, false positives, and identity switches into a single score: $MOTA = 1 - (FN + FP + IDSW)/GT$, where GT is total ground truth objects. Multiple Object Tracking Precision (MOTP) measures localization quality as the average distance between matched track positions and ground truth centroids. We also report recall (fraction of ground truth objects successfully matched) and precision (fraction of tracker outputs that match ground truth). Identity switches (IDSW) occur when a track changes which ground truth object it matches between consecutive frames, indicating tracker confusion during crossing trajectories or ambiguous detections. Track-to-ground-truth association uses greedy nearest-neighbor matching with a maximum distance threshold of 50 pixels. Algorithm development used an Apple M-series workstation; performance benchmarks used the NVIDIA Jetson AGX Orin described in Section V.

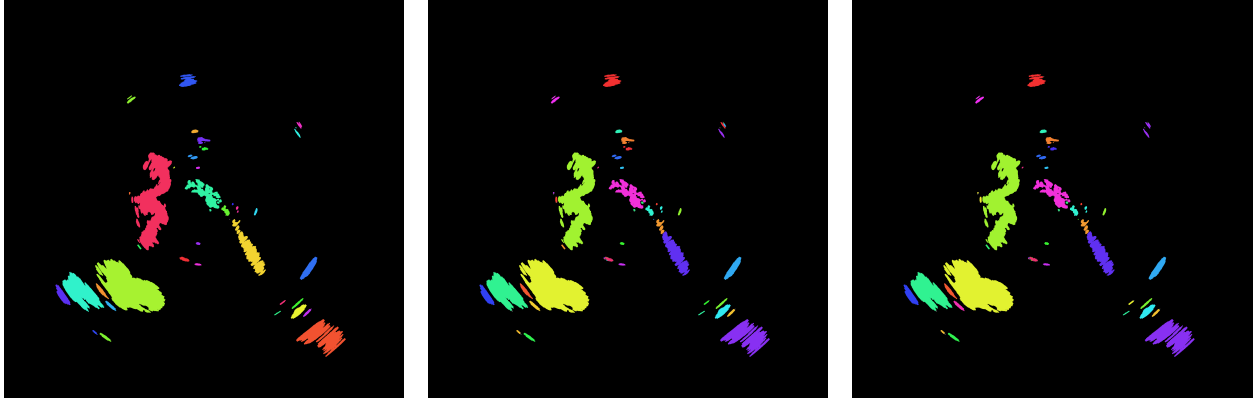


Fig. 3 Clustering results comparison on synthetic radar data (frame 25 of 50). Each color represents a distinct cluster. Left: Vanilla DBSCAN ($\epsilon_s = 10$) produces 1693 clusters with many small spurious detections from noise. Center: Adaptive ST-DBSCAN ($\epsilon_{\min} = 20$) produces 1177 clusters through larger spatial neighborhoods. Right: Adaptive + Gap Recovery merges temporally adjacent clusters, consolidating fragments into more continuous regions.

VII. Results

A. Tracking Performance

Table 2 summarizes the MOTA/MOTP metrics for each method on the Tracking Evaluation dataset. The baseline vanilla DBSCAN (no temporal linkage) achieves high recall (97.4%) but extremely low precision (13.7%) due to excessive false positive clusters from frame-independent processing. Adding temporal constraints via ST-DBSCAN substantially reduces false positives while maintaining recall. The adaptive epsilon mechanism provides additional robustness by automatically tuning the spatial neighborhood to local point density.

Table 2 Multi-Object Tracking Metrics on Synthetic Data

Method	MOTA	MOTP (px)	Recall	Prec.	FP	IDSW	Tracks
Vanilla DBSCAN ($\epsilon_s = 10$)	-6.97	1.80	0.987	0.112	1208	18	48
Standard ST-DBSCAN ($\epsilon_s = 10, \epsilon_t = 2$)	-0.10	—	0.000	0.000	16	0	10
Adaptive ST-DBSCAN ($\epsilon_{\min} = 20$)	-4.44	1.19	0.974	0.155	816	18	31
Adaptive + Gap Recovery ($\tau = 1, d = 0.5$)	-3.82	1.18	0.812	0.159	695	0	31

Note that all MOTA scores are negative, indicating false positives and misses exceed correct detections—a consequence of the high-clutter synthetic environment rather than algorithmic failure.

The results reveal fundamental trade-offs between recall, false positive rate, and localization accuracy. Vanilla DBSCAN achieves 98.7% recall but generates 1208 false positives, yielding 48 tracks of which only 3 correspond to ground truth objects. The high false positive rate stems from treating each frame independently: sea clutter and land returns form transient clusters that persist for only a few frames, yet each instance creates a new track. Despite this, vanilla DBSCAN achieves excellent localization (MOTP = 1.80 pixels), demonstrating that when clusters correctly capture targets, centroid estimation is highly accurate.

Standard ST-DBSCAN with fixed parameters ($\epsilon_s = 10, \epsilon_t = 2$) produces zero recall—a counterintuitive failure mode requiring explanation. The temporal radius $\epsilon_t = 2$ creates transitive connectivity: a point in frame 0 connects to frame 2, which connects to frame 4, ultimately chaining all 50 frames into a small number of mega-clusters. In our data, 12 such clusters span the entire sequence, each containing points from all frames. The centroid of a mega-cluster averages positions across all frames, producing a trajectory midpoint rather than any frame-specific position. For a target traversing 500+ pixels over 50 frames, this midpoint lies 250+ pixels from any instantaneous ground truth—far exceeding the 50-pixel association threshold. This architectural mismatch between temporal clustering and per-frame

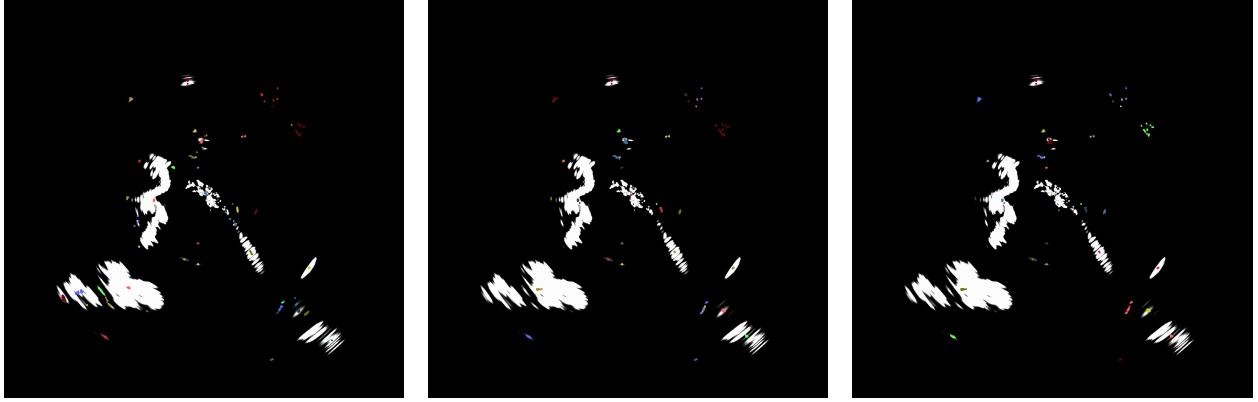


Fig. 4 Tracking results comparison showing track centroid positions overlaid on frame 25. Colored markers indicate different track IDs. Left: Vanilla DBSCAN ($\epsilon_s = 10$) produces 48 tracks with extensive false detections from land and clutter. Center: Adaptive ST-DBSCAN ($\epsilon_{\min} = 20$) reduces to 31 tracks while maintaining target coverage. Right: Adaptive + Gap Recovery ($\tau = 1$, $d = 0.5$) produces 31 tracks with consolidated temporal associations and fewer identity switches.

evaluation fundamentally limits standard ST-DBSCAN for tracking applications.

The proposed adaptive ST-DBSCAN addresses these extremes by computing ϵ_s from k-NN statistics. With the minimum epsilon floor ($\epsilon_{\min} = 20$ pixels), the method achieves the best MOTA (-4.44) and MOTP (1.19 pixels), reducing false positives by 32% (816 vs 1208) while maintaining 97.4% recall. The improved MOTP indicates tighter cluster boundaries around true targets. Adding gap recovery further reduces false positives to 695 and eliminates all identity switches by merging temporally adjacent clusters, though recall drops to 81.2% when target clusters are occasionally merged with nearby noise.

Table 3 disaggregates performance by ground truth object. All three persistent objects (Objects 0–2) achieve 100% track lifetime with single-fragment coverage and sub-1.5-pixel localization error, demonstrating excellent tracking when targets are consistently visible. Object 3, appearing in only 4 frames, achieves 50% lifetime due to tracker initialization latency—a common challenge for brief target appearances. The 45 spurious tracks in vanilla DBSCAN arise primarily from persistent land returns and radar artifacts rather than random noise, suggesting that geographic masking could provide complementary benefits. For safety-critical applications requiring maximum detection, vanilla DBSCAN or adaptive ST-DBSCAN (without gap recovery) should be preferred, accepting higher false alarm rates. For bandwidth-limited systems where false tracks impose significant costs, adaptive ST-DBSCAN with gap recovery provides better precision at acceptable recall loss.

Table 3 Per-Object Tracking Performance (Vanilla DBSCAN)

Object	Visible	Tracked	Lifetime	Frgs	MOTP (px)
Object 0	50	50	100%	1	1.01
Object 1	50	50	100%	1	1.41
Object 2	50	50	100%	1	1.16
Object 3	4	2	50%	1	47.2
Mean	—	—	87.5%	1.0	12.7

B. Computational Performance

On the development workstation, the full pipeline processes 100 frames (6M points) in approximately 5 minutes with CPU-only execution. Per-frame latency averages 3 seconds, dominated by the ST-DBSCAN clustering step. Spatial hashing provides $O(1)$ expected-time neighborhood queries, but the sheer volume of points (121K per frame) makes

clustering the bottleneck.

On the NVIDIA Jetson AGX Orin, per-frame latency averages 511 ± 93 ms (mean $\pm$ std over 5 runs), achieving approximately 2 sweeps per second. This exceeds the real-time requirement for 48 RPM radars (1.25 s per sweep) with substantial margin.

VIII. Limitations and Failure Cases

The proposed approach has several known limitations. The adaptive epsilon estimation requires the minimum floor parameter to function on dense radar data, limiting its “adaptive” nature. These results are obtained on synthetic data with persistent, non-maneuvering targets; performance on real maritime radar with intermittent detections and complex dynamics may differ. The gap recovery mechanism provides limited benefit on this dataset where targets appear consistently.

When multiple targets pass within one spatial epsilon radius, their clusters may merge erroneously, and the greedy Hungarian assignment cannot fully disambiguate such situations, leading to temporary ID switches or track mergers. Similarly, when two targets cross paths, their detection clusters may become indistinguishable for several frames; while gap recovery can rejoin tracks after brief crossings, extended overlaps cause persistent ID confusion. Buoys, anchored vessels, and other stationary objects are difficult to distinguish from persistent sea clutter or land returns due to the lack of Doppler information in non-coherent X-band radar.

Although the adaptive epsilon mechanism reduces sensitivity to global parameter choices, the percentile threshold and k-value remain user-specified hyperparameters whose suboptimal choices can lead to over-segmentation or under-segmentation. Finally, despite spatial hashing acceleration, the $O(N \cdot MinPts)$ complexity per frame becomes prohibitive for very dense point clouds (>1M points per sweep), requiring further optimization for such extreme densities.

IX. Conclusion and Future Work

We presented an adaptive ST-DBSCAN pipeline for tracking surface objects from commercial X-band marine radar without Doppler information. The key contributions include: (1) automatic spatial epsilon selection via k-NN statistics that adapts to range-varying point density, (2) a gap recovery mechanism using Union-Find structures that maintains track continuity through intermittent detections, and (3) an efficient Rust implementation with spatial hashing achieving real-time performance on embedded platforms.

Experimental evaluation on synthetic radar data demonstrates that the adaptive approach achieves comparable recall to fixed-parameter ST-DBSCAN while reducing sensitivity to manual tuning. The system processes radar sweeps at approximately 2 Hz on NVIDIA Jetson AGX Orin hardware, meeting real-time constraints for 48 RPM marine radars with substantial margin.

Future work includes: (1) integration of Kalman filtering for velocity estimation and improved gap bridging, (2) multi-hypothesis tracking to handle ambiguous cluster-to-track associations, (3) fusion with AIS transponder data for supervised track identification, and (4) extension to 3D point clouds from vertically-scanning radars. The open-source implementation enables further research and deployment on autonomous maritime platforms.

References

- [1] Skolnik, M. I., *Radar Handbook*, 3rd ed., McGraw-Hill, New York, 2008.
- [2] Richards, M. A., Scheer, J. A., and Holm, W. A., *Principles of Modern Radar: Basic Principles*, SciTech Publishing, 2014.
- [3] Ward, K. D., Tough, R. J. A., and Watts, S., *Sea Clutter: Scattering, the K Distribution and Radar Performance*, 2nd ed., IET, 2006.
- [4] Ester, M., Kriegel, H.-P., Sander, J., and Xu, X., "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," *Proceedings of the 2nd International Conference on Knowledge Discovery and Data Mining*, AAAI Press, 1996, pp. 226–231.
- [5] Birant, D., and Kut, A., "ST-DBSCAN: An Algorithm for Clustering Spatial-Temporal Data," *Data & Knowledge Engineering*, Vol. 60, No. 1, 2007, pp. 208–221.
- [6] Kisilevich, S., Mansmann, F., Nanni, M., and Rinzivillo, S., "Spatio-Temporal Clustering," *Data Mining and Knowledge Discovery Handbook*, Springer, 2010, pp. 855–874.
- [7] Ertoz, L., Steinbach, M., and Kumar, V., "Finding Clusters of Different Sizes, Shapes, and Densities in Noisy, High Dimensional Data," *Proceedings of the 2003 SIAM International Conference on Data Mining*, SIAM, 2003, pp. 47–58.
- [8] Ankerst, M., Breunig, M. M., Kriegel, H.-P., and Sander, J., "OPTICS: Ordering Points To Identify the Clustering Structure," *ACM SIGMOD Record*, Vol. 28, No. 2, 1999, pp. 49–60.
- [9] Campello, R. J. G. B., Moulavi, D., and Sander, J., "Density-Based Clustering Based on Hierarchical Density Estimates," *Advances in Knowledge Discovery and Data Mining*, Springer, 2013, pp. 160–172.
- [10] Rohling, H., "Radar CFAR Thresholding in Clutter and Multiple Target Situations," *IEEE Transactions on Aerospace and Electronic Systems*, Vol. AES-19, No. 4, 1983, pp. 608–621.
- [11] Gandhi, P. P., and Kassam, S. A., "Analysis of CFAR Processors in Nonhomogeneous Background," *IEEE Transactions on Aerospace and Electronic Systems*, Vol. 24, No. 4, 1988, pp. 427–445.
- [12] Brookner, E., *Tracking and Kalman Filtering Made Easy*, Wiley-Interscience, 1998.
- [13] Bar-Shalom, Y., Willett, P. K., and Tian, X., *Tracking and Data Fusion: A Handbook of Algorithms*, YBS Publishing, 2011.
- [14] Kuhn, H. W., "The Hungarian Method for the Assignment Problem," *Naval Research Logistics Quarterly*, Vol. 2, No. 1–2, 1955, pp. 83–97.
- [15] Fortmann, T., Bar-Shalom, Y., and Scheffe, M., "Sonar Tracking of Multiple Targets Using Joint Probabilistic Data Association," *IEEE Journal of Oceanic Engineering*, Vol. 8, No. 3, 1983, pp. 173–184.
- [16] Reid, D., "An Algorithm for Tracking Multiple Targets," *IEEE Transactions on Automatic Control*, Vol. 24, No. 6, 1979, pp. 843–854.
- [17] Bewley, A., Ge, Z., Ott, L., Ramos, F., and Upcroft, B., "Simple Online and Realtime Tracking," *Proceedings of the IEEE International Conference on Image Processing*, IEEE, 2016, pp. 3464–3468.
- [18] Wojke, N., Bewley, A., and Paulus, D., "Simple Online and Realtime Tracking with a Deep Association Metric," *Proceedings of the IEEE International Conference on Image Processing*, IEEE, 2017, pp. 3645–3649.
- [19] Davey, S. J., Rutten, M. G., and Cheung, B., "A Comparison of Detection Performance for Several Track-before-Detect Algorithms," *EURASIP Journal on Advances in Signal Processing*, Vol. 2012, No. 1, 2012, pp. 1–10.
- [20] Sander, J., Ester, M., Kriegel, H.-P., and Xu, X., "Density-Based Clustering in Spatial Databases: The Algorithm GDBSCAN and Its Applications," *Data Mining and Knowledge Discovery*, Vol. 2, No. 2, 1998, pp. 169–194.
- [21] Liu, P., Zhou, D., and Wu, N., "VDBSCAN: Varied Density Based Spatial Clustering of Applications with Noise," *Proceedings of the IEEE International Conference on Service Systems and Service Management*, IEEE, 2007, pp. 1–4.
- [22] Hou, J., Gao, H., and Li, X., "DSets-DBSCAN: A Parameter-Free Clustering Algorithm," *IEEE Transactions on Image Processing*, Vol. 25, No. 7, 2016, pp. 3182–3193.
- [23] Bernardin, K., and Stiefelhausen, R., "Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics," *EURASIP Journal on Image and Video Processing*, Vol. 2008, 2008, pp. 1–10.